THREADED BST PREDECESSOR AND SUCCESSOR

Problem Statement:

Develop a C++ program to build a threaded binary search tree (BST) and display the inorder predecessor and successor of a specified node as well as every node. The program should handle the minimum and maximum nodes that lack a predecessor or successor, and verify all results against the inorder traversal sequence.

Objective:

The objective of this project is to implement a threaded binary search tree in C++ and find the inorder predecessor and successor of every node using threads, without recursion or a stack. It also handles the minimum and maximum nodes that have no predecessor or successor, and verifies all results against the inorder traversal sequence, while strengthening understanding of BST insertion, searching and traversal.

Algorithm:

Step 1: Start the program and set root = NULL.

Step 2: Define a node with data, left, right, lthread and rthread. A new node has both flags true and both pointers NULL.

Step 3: Read a key to insert. If the tree is empty, make it the root. Otherwise, move left if key < data or right if key > data until the next link is a thread, and ignore duplicate keys.

Step 4: If the key is smaller than the parent, set new->left = parent->left, new->right = parent, parent->left = new and parent->lthread = false. For a larger key, do the mirror steps on the right side.

Step 5: Successor: if rthread = true, it is node->right (NULL for the maximum node). Otherwise, go to the right child and keep moving left until lthread = true.

Step 6: Predecessor: if lthread = true, it is node->left (NULL for the minimum node). Otherwise, go to the left child and keep moving right until rthread = true.

Step 7: Find the minimum node by moving left until lthread = true, and the maximum node by moving right until rthread = true.

Step 8: For a specified node, read the key and search for it using only real child links. If it is not found, display "Node not found".

Step 9: If found, display its predecessor and successor. Display "None (minimum node)" or "None (maximum node)" where the value is NULL.

Step 10: For every node, start at the minimum node and display each node with its predecessor and successor, moving to the successor until NULL.

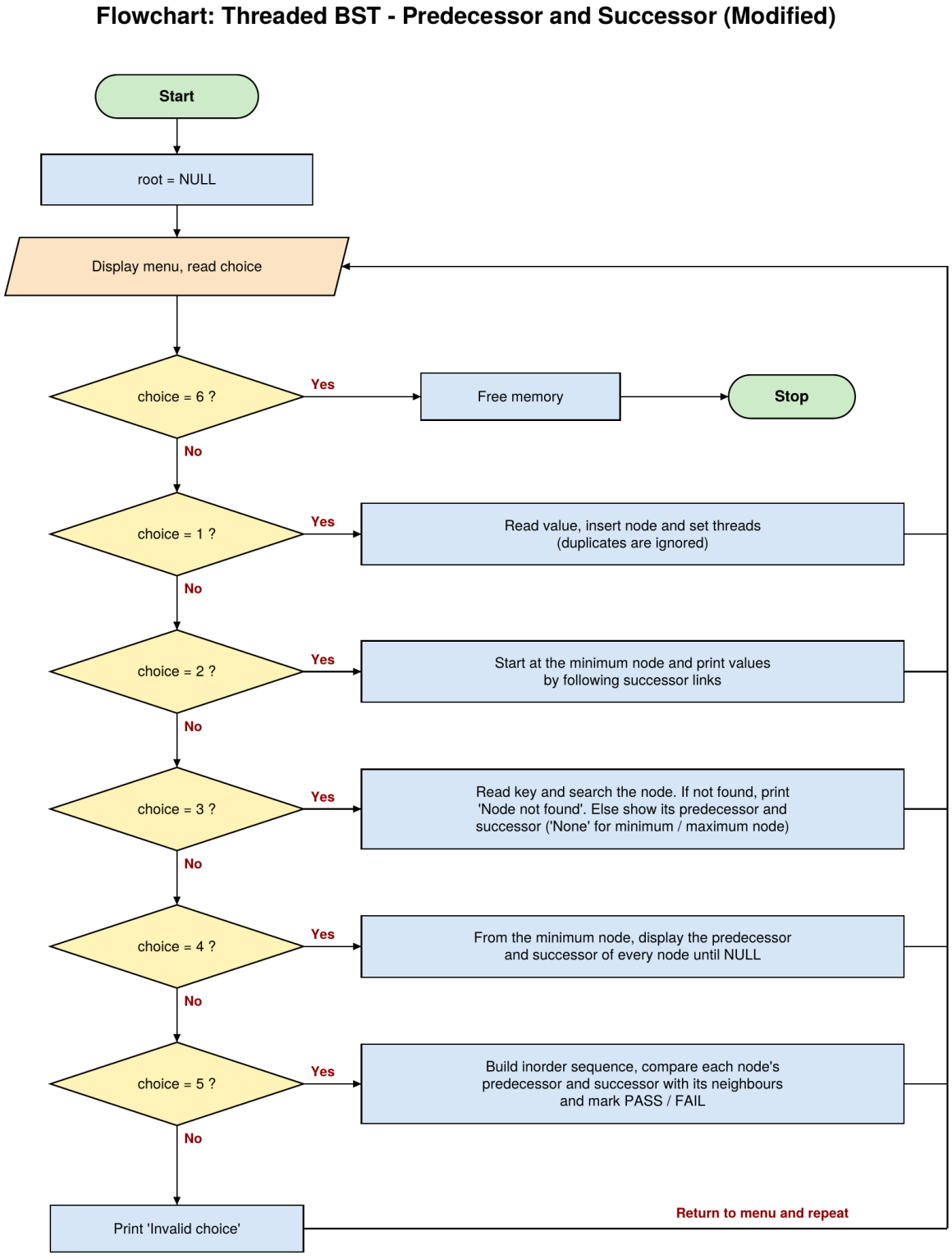
Step 11: Build the inorder sequence by storing the values in order, starting from the minimum node and following successor links.

Step 12: For each position i in the sequence, the expected predecessor is seq[i-1] (none for the first) and the expected successor is seq[i+1] (none for the last).

Step 13: Compare the thread-based results with the expected values, mark each node PASS or FAIL, and display the number of matched nodes.

Step 14: Repeat the menu until the user exits, then free the allocated memory and stop the program.

Flowchart:



Program: threaded_bst_modified.cpp (menu-driven, double-threaded BST)